

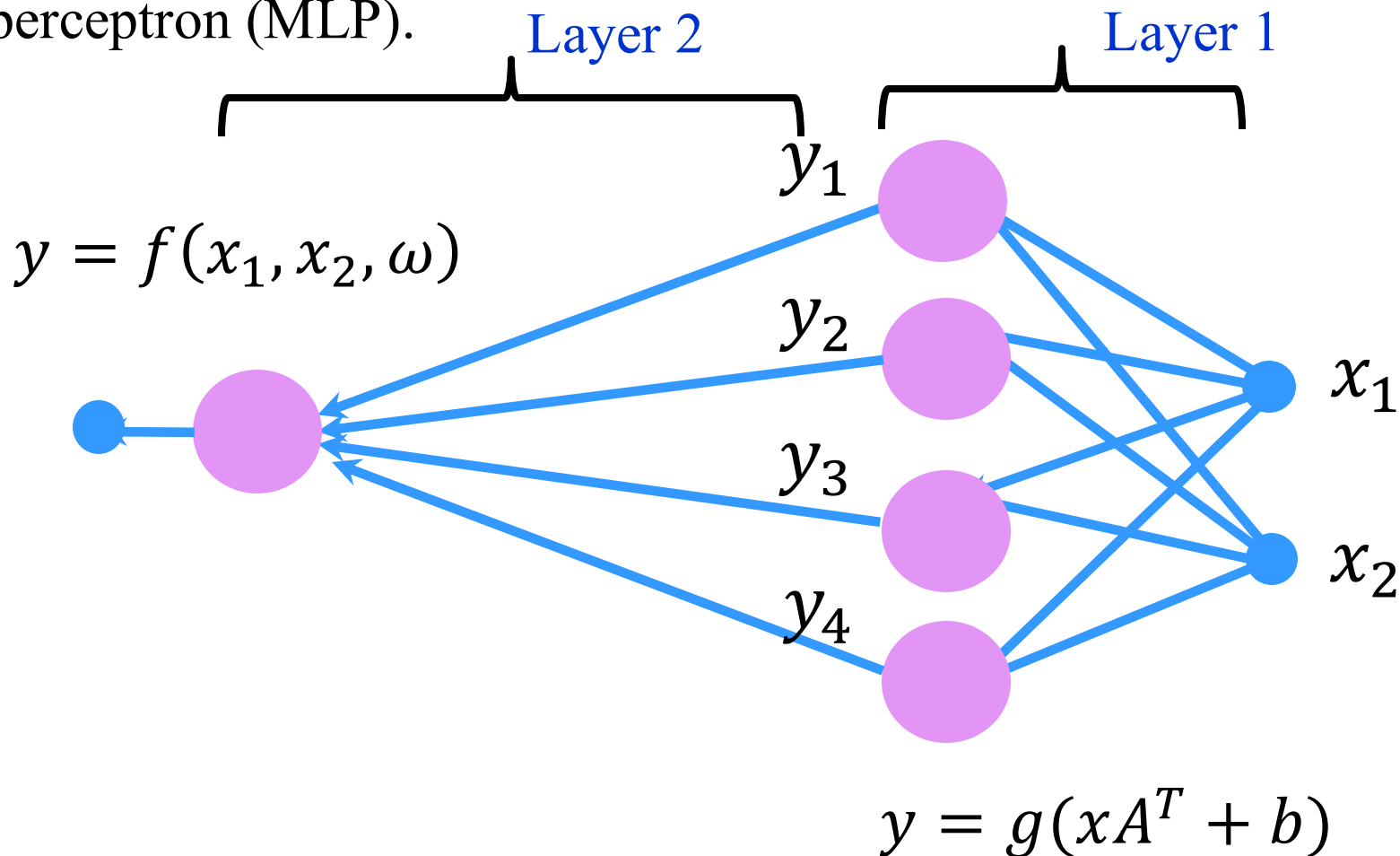
MACHINE LEARNING IN PHYSICS FOUNDATIONS 2

HARRISON B. PROSPER

PHY6937 / PHY4936

Recap

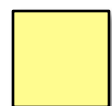
Last time we introduced the multi-node (and multi-layer) perceptron (MLP).



Recap

We also noted that ML models are usually drawn using high-level components. For example:

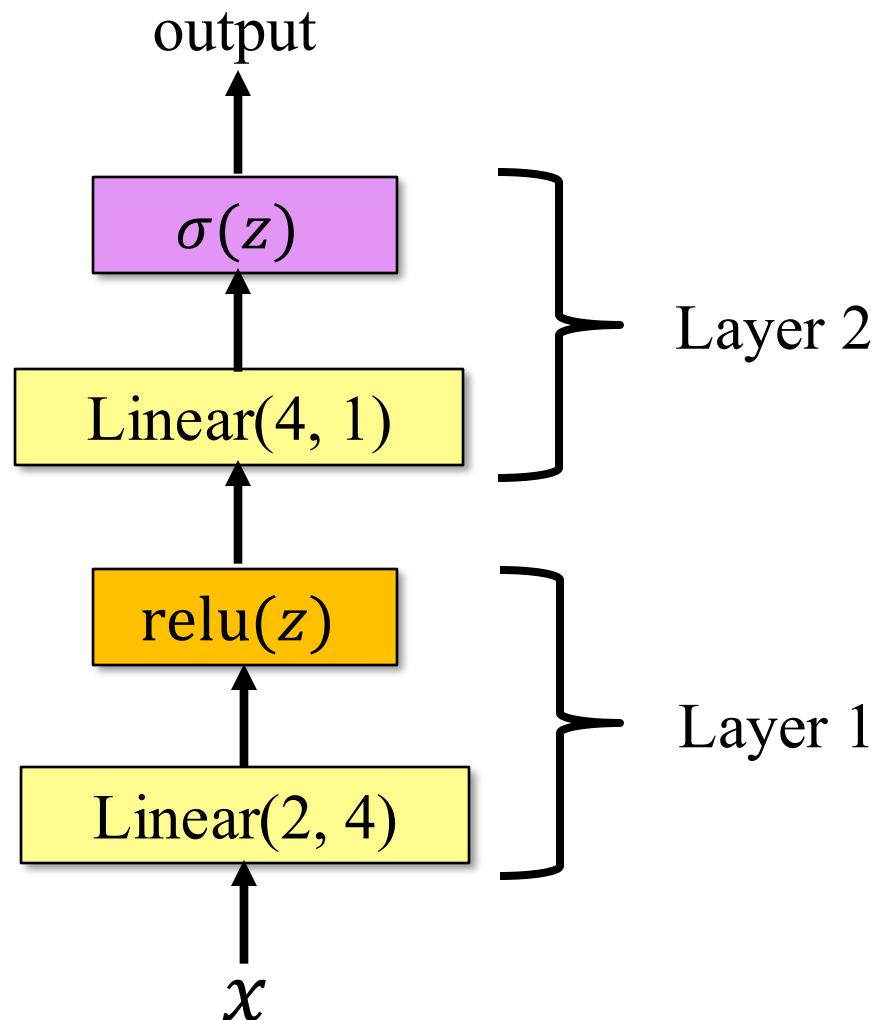
$$f(z) = zA^T + b$$



$$g(z) = \text{relu}(z)$$



$$\sigma(z) = (1 + e^{-z})^{-1}$$



FITTING MODELS TO DATA

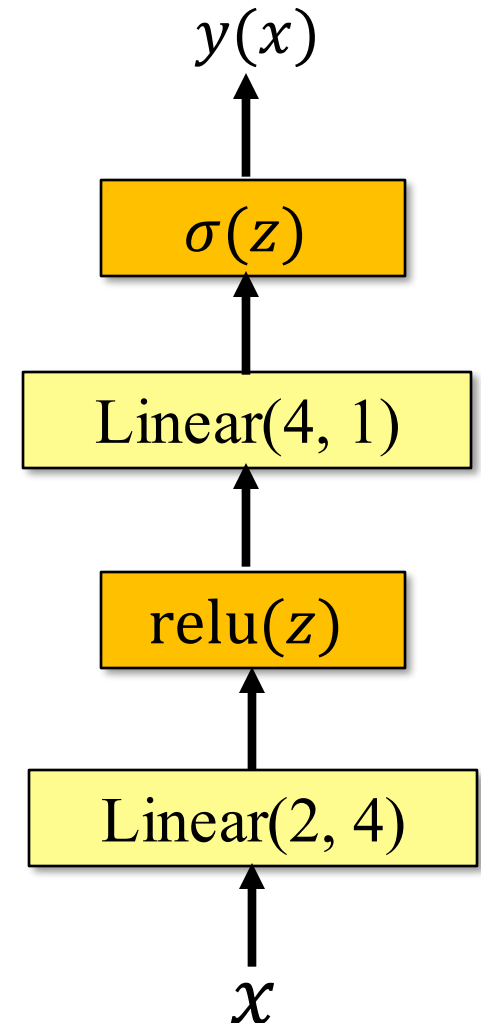
Fitting Models to Data (1)

Suppose our goal is to approximate an unknown function

$$y = f(x_1, x_2)$$

given a dataset $D = \{(x_1, x_2, y)_{i=1}^N\}$.

The standard approach is to approximate $f(x_1, x_2)$ with a parameterized function $f(x_1, x_2, \omega)$, that is, a model, where ω are the parameters of the model.



Fitting Models to Data (2)

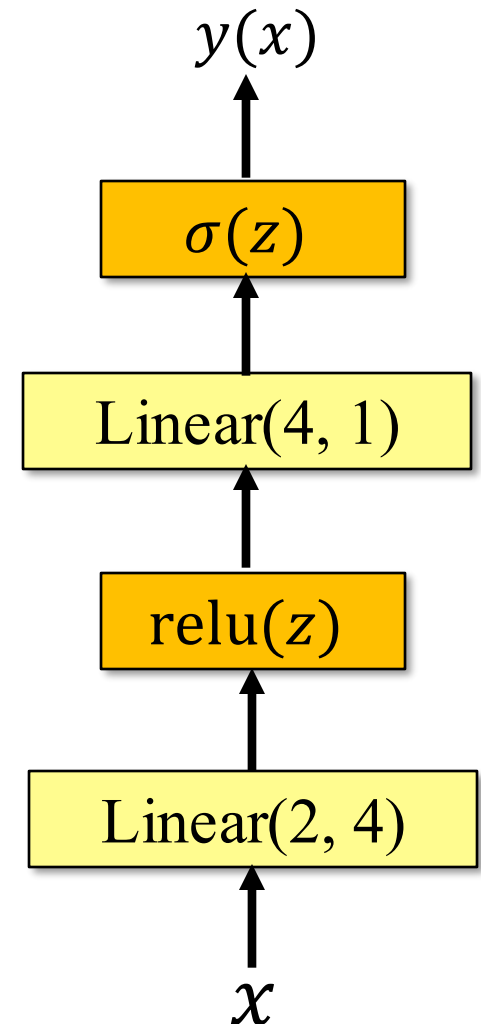
One way to find the *best-fit values* ω^* of the model $f(x_1, x_2, \omega)$ is to minimize the quadratic form

$$R(\omega) = \frac{1}{N} \sum_{i=1}^N (y_i - f_i)^2$$

with respect to the parameters ω , where

$$f_i = f(x_{1i}, x_{2i}, \omega).$$

This is called a **least-squares fit**.



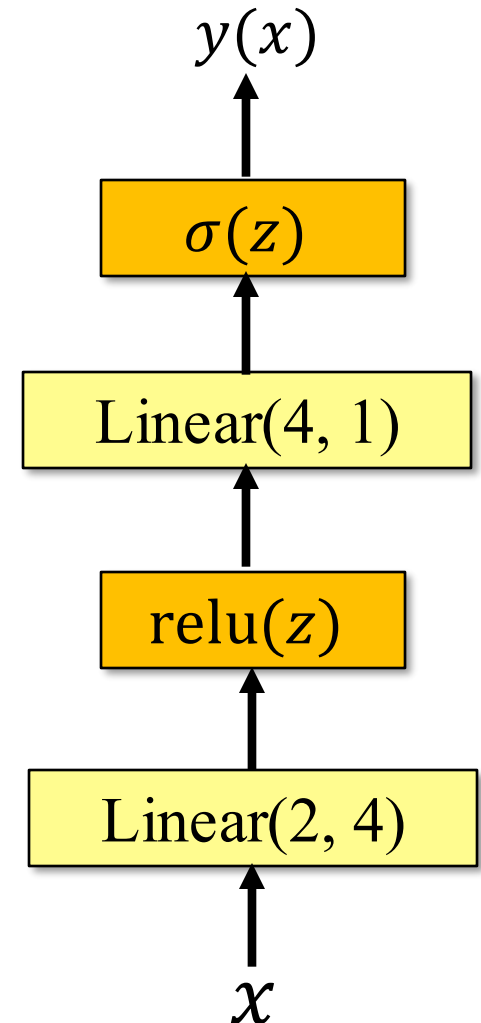
Fitting Models to Data (3)

The least-squares fit can be generalized to

$$R(\omega) = \frac{1}{N} \sum_{i=1}^N L(y_i, f_i)$$

where, $L(y_i, f_i)$, the **loss function**, measures the discrepancy between the desired **target** y and the model, f .

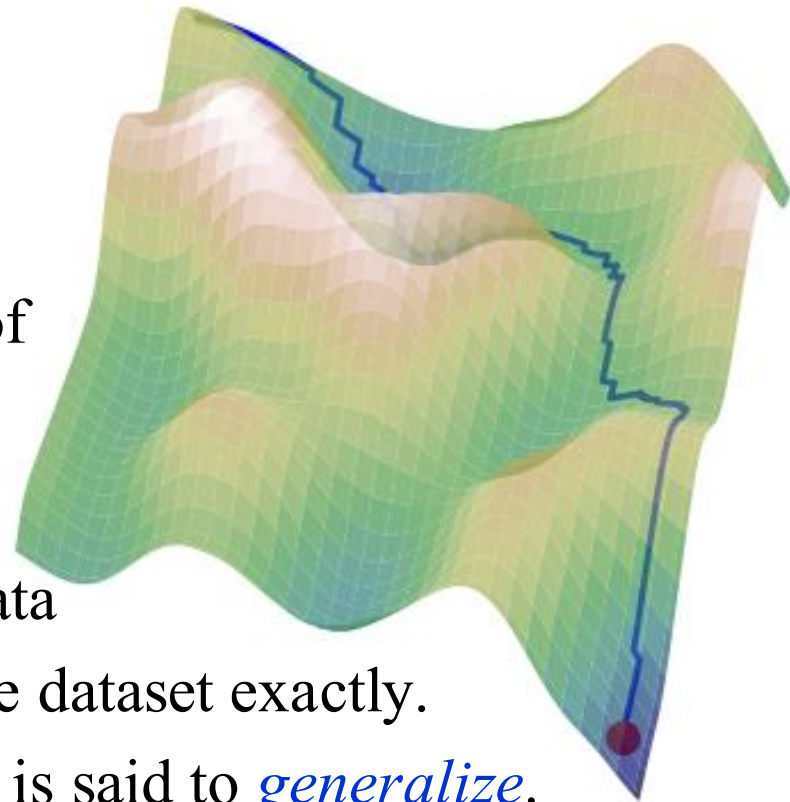
$R(\omega)$ is called the **empirical risk** or average loss. **Warning:** In ML, the empirical risk is often referred to as the “loss”.



Empirical Risk Landscape (1)

The empirical risk defines a highly corrugated high-dimensional “landscape” in the space of parameters.

Our task is to navigate the empirical risk landscape to find a *good approximation* of the minimum not of that landscape but rather of the landscape that would be formed if we if we had an *infinite* amount of data and a model that would fit the infinite dataset exactly. A model that approximates this ideal is said to *generalize*.



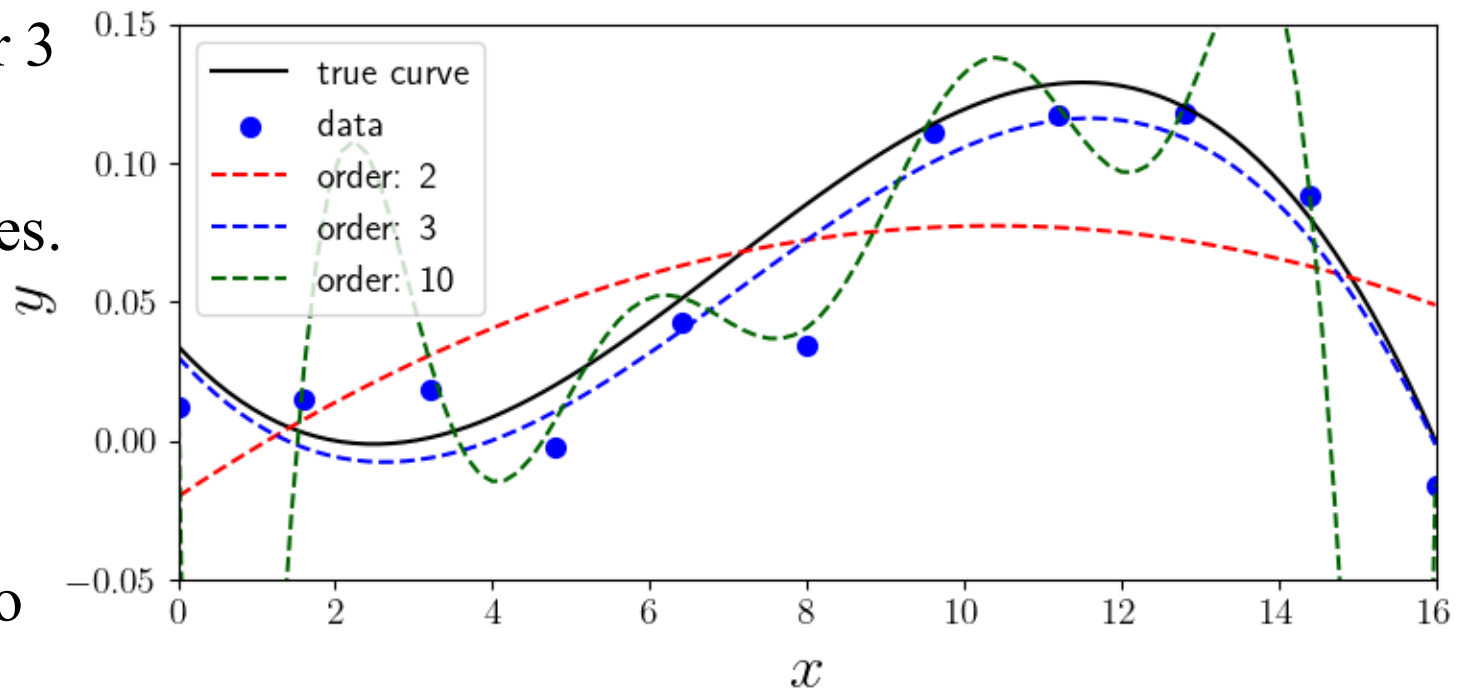
Empirical Risk Landscape (2)

Example: A 1D least squares fit to a **linear model**.

$$f(x, \omega) = \sum_{j=0}^n \omega_j g_j(x), \quad g_j(x) = x^j$$

The order 3
model
generalizes.

But the
other
models do
not.



RISK FUNCTIONAL

Risk Functional (1)

In mathematics and physics, insight can often be gained by taking some limit of an expression. Consider the limit of

$$R(\omega) = \frac{1}{N} \sum_{i=1}^N L(y_i, f_i)$$

as $N \rightarrow \infty$, that is, as the amount of data grows without limit.

In that limit, the empirical risk becomes the **risk functional**,

$$R[f] = \int dx \int dy L(y, f) p(x, y)$$

where $p(x, y)dx dy$ is the probability distribution of the data.

Risk Functional (2)

To find the optimal function $f = f^*$, ideally, we should minimize the risk functional

$$R[f] = \int dx \int dy L(y, f) p(x, y)$$

While we know the functional form of the loss function $L(y, f)$ because we choose it, the probability distribution, $p(x, y)dx dy$, of the data is usually unknown.

However, because $R(\omega)$ is a **Monte Carlo** approximation of $R[f]$, any useful result derived for the risk functional pertains, at least approximately, to the empirical risk.

Risk Functional (3)

To minimize

$$R[f] = \int dx \int dy L(y, f) p(x, y)$$

first note that $p(x, y) = p(y|x) p(x)$.

Therefore, we can write the functional $R[f]$ as

$$R[f] = \int dx p(x) \mathcal{L}(x, f)$$

where,

$$\mathcal{L}(x, f) = \int dy L(y, f) p(y|x)$$

Risk Functional (4)

Add an arbitrary function $\epsilon g(x)$ to the optimal function f^* , where ϵ is an infinitesimal*.

$$\begin{aligned} R[f^* + \epsilon g] &= \int dx \, p(x) \, \mathcal{L}(x, f^* + \epsilon g) \\ &= \int dx \, p(x) \left(\mathcal{L}(x, f^*) + \epsilon g \frac{\partial \mathcal{L}}{\partial f^*} \right) \\ &= R[f^*] + \epsilon \int dx \, p(x) g(x) \frac{\partial \mathcal{L}}{\partial f^*} \end{aligned}$$

* $0 < |\epsilon| < |x|$, where x is any real number.

Risk Functional (5)

Rearranging, and taking standard part* of both sides, we find

$$\text{st}\left(\frac{R[f^* + \epsilon g] - R[f^*]}{\epsilon}\right) = \int dx p(x) g(x) \frac{\partial \mathcal{L}}{\partial f^*}$$

The lefthand side defines the functional derivative $\delta R / \delta f$.

By assumption, $\delta R / \delta f$ is zero at $f = f^*$, $\forall x$. Therefore,

$$\int dx p(x) g(x) \frac{\partial \mathcal{L}}{\partial f} = 0$$

when $f = f^*$.

The standard part of the hyperreal $x^ = x + \epsilon a$ is defined by $x = \text{st}(x^*)$

Risk Functional (6)

We want the expression

$$\int dx p(x) g(x) \frac{\partial \mathcal{L}}{\partial f} = 0$$

to hold for *any* function $g(x)$ and $\forall x$.

This can happen *if only if*

$$\frac{\partial \mathcal{L}}{\partial f} = 0$$

Assuming the integral and partial derivative operations commute, and recalling that $\mathcal{L}(\mathbf{x}, \mathbf{f}) = \int dy L(y, \mathbf{f}) p(y | \mathbf{x})$, we arrive at the

Very Important Result:

$$\int dy \frac{\partial L}{\partial f} p(y | \mathbf{x}) = 0$$

Risk Functional (7)

Points to Note:

$$\int dy \frac{\partial L}{\partial f} p(y | x) = 0$$

1. This result is independent of the details of $f(x, \omega)$.
However,...
2. The function $f(x, \omega)$ must have sufficient **capacity**: i.e., there must exist an approximation $f(x, \hat{\omega})$ found by minimizing the empirical risk $R(\omega)$ that is arbitrarily close to the optimal function f^* .
3. Moreover, it must be possible to find that function!

EMPIRICAL RISK MINIMIZATION

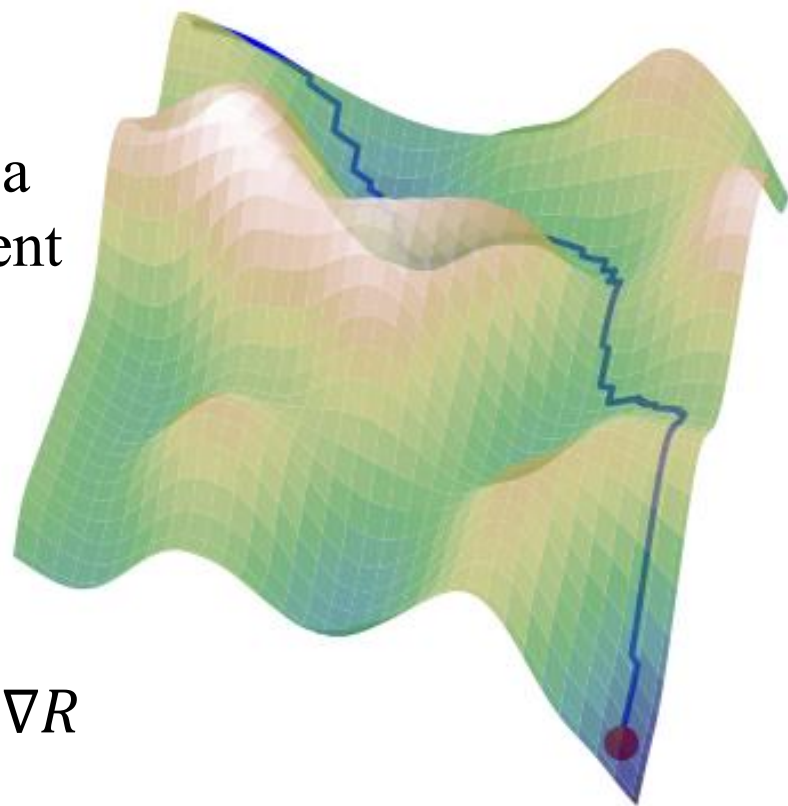
Empirical Risk Minimization (1)

The minimization of $R(\omega)$ is typically done by moving in the direction of *steepest descent*. Most optimization algorithms in ML are variations of **Stochastic Gradient Descent**.

Algorithm

1. At the current point ω_j , compute a *noisy* approximation of the gradient of $R(\omega) \approx \frac{1}{n} \sum_{i=1}^n L(y_i, f_i)$ by using a **batch** of **training data**, where $n \ll N$.
2. Move to the next position ω_{j+1} in the landscape using

$$\omega_{j+1} = \omega_j - \eta \nabla R$$



Empirical Risk Minimization (2)

Why does this (1st-order) algorithm

$$\omega_{j+1} = \omega_j - \eta \nabla R$$

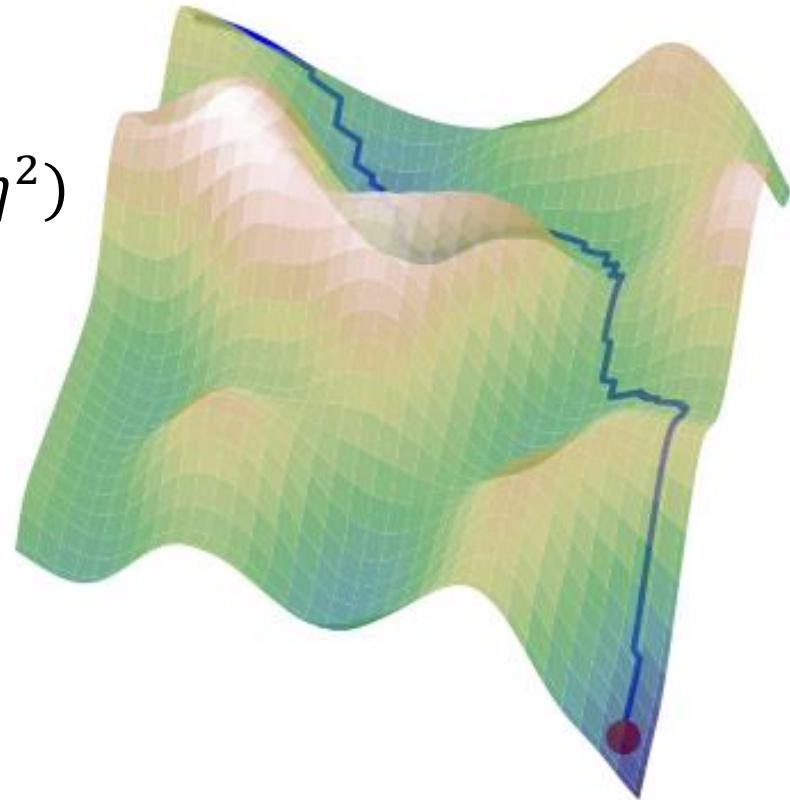
work?

Consider

$$\begin{aligned} R(\omega_{j+1}) &= R(\omega_j - \eta \nabla R) \\ &= R(\omega_j) - \eta \nabla R \cdot \nabla R + O(\eta^2) \end{aligned}$$

If the $O(\eta^2)$ can be neglected, and given that the $O(\eta)$ term is always negative, it follows that

$$R(\omega_{j+1}) < R(\omega_j).$$



COMMON LOSS FUNCTIONS

Common Loss Functions (1)

Quadratic loss (often used for regression)

$$L(y, f) = (y - f)^2$$

Binary cross entropy (often used for classification)

$$L(y, f) = -[y \log f + (1 - y) \log(1 - f)]$$

Exponential loss

$$L(y, f) = \exp(-wyf/2)$$

Quantile loss ($0 \leq \tau \leq 1$)

$$L(y, f) = \begin{cases} \tau(y - f) & y \geq f \\ (1 - \tau)(f - y) & y < f \end{cases}$$

Zero-One loss

$$L(y, f) = \begin{cases} 0 & |y - f| \leq b \\ 1 & |y - f| > b \end{cases}$$

Common Loss Functions (2)

Quadratic loss: $L(y, f) = (y - f)^2$

$$\int \frac{\partial L}{\partial f} p(y|x) dy = 0$$

Solution

$$f(x, \omega^*) = \int y p(y | x) dy$$

Very Important Point (VIP): As noted, the result is independent of the details of f . The result depends solely on the form of the loss function and the conditional probability, $p(y|x)dy$, of the data.

Binary Cross Entropy Loss (1)

Binary cross entropy loss : $-[y \log f + (1 - y) \log(1 - f)]$

$$\int \frac{\partial L}{\partial f} p(y|x) dy = 0$$

$$\int \left[\frac{y}{f} - \frac{(1 - y)}{1 - f} \right] p(y|x) dy = 0$$

Note: $y = \{0, 1\}$. Therefore,

$$-\frac{p(y = 0|x)}{1 - f} + \frac{p(y = 1|x)}{f} = 0,$$

which implies

$$f(x, \omega^*) = p(y = 1|x)$$

Binary Cross Entropy Loss (2)

The result

$$f(x, \omega^*) = p(1|x) = \frac{p(x|1) \pi(1)}{p(x)}$$

with $p(x) = p(x|1) \pi(1) + p(x|0) \pi(0)$, ($k \equiv y = k$), shows that any model trained using the the cross-entropy loss necessarily approximates a **class probability**, that is, the probability that an object characterized by the **features** (i.e., attributes) x belongs to the class labeled $y = 1$.

- $p(x|k)$ is the probability density of x for class $y = k$.
- $\pi(k)$ is the prior probability of class $y = k$.

Binary Cross Entropy Loss (3)

Example: A Spam Filter

A spam filter is a binary classifier: spam vs. non-spam emails. Suppose $y = 1$ and $y = 0$ labels spam and non-spam emails, respectively, and the training dataset comprises 50% spam emails and 50% non-spam emails. That is, the dataset is **balanced**.

- Each email is characterized by a vector x , and $p(x|1)$ and $p(x|0)$ are the probability densities of these vectors for spam and non-spam emails, respectively.
- The prior probabilities are $\pi(1) = 0.50$ and $\pi(0) = 0.50$.

The Discriminant

When the training dataset is balanced, the classification probability $p(1|x)$, which in physics is often refer to as a **discriminant**, becomes

$$D(x) = \frac{p(x|1)}{p(x|1) + p(x|0)}$$

This is one of the most useful results in machine learning.

The next couple of slides offer some examples of how useful a result this is.

The Discriminant: Uses (1)

The discriminant suggests a way to model a probability density, given a known density. First note that

$$D(x) = \frac{p(x|1)}{p(x|1) + p(x|0)}$$

can be rewritten as

$$p(x|1) = \left[\frac{D(x)}{1 - D(x)} \right] p(x|0)$$

using the so-called likelihood ratio trick. If $p(x|0)$ is a known density, from which the class $y = 0$ vectors are sampled, then the discriminant can be repurposed as a **probability density estimator**.

The Discriminant: Uses (2)

It is of course possible to train a classifier using an unbalanced dataset: $\pi(1) \neq \pi(0)$. We've just shown that the model will approximate the classification probability $p(1|x)$.

But what if $\pi(1) \ll \pi(0)$? The classifier is unlikely to be very good. (Why do you think this is?)

Instead, train with a balanced dataset and compute $p(1|x)$ using

$$p(1|x) = \frac{D(x)\pi(1)}{D(x)\pi(1) + [1 - D(x)]\pi(0)}$$

where

$$D(x) = p(x|1)/[p(x|1) + p(x|0)]$$

Summary (1)

Stochastic Gradient Descent

- Many (1st-order) minimization algorithms are variations of the following algorithm:

$$\omega_{j+1} = \omega_j - \eta \nabla R$$

- Batches of data are used, which introduces noise into ∇R .

Risk Minimization

- Given a data set $D = \{(x, y)\}_{i=1}^N$, a model $f(x, \omega)$, and a loss function $L(y, f)$, the optimal function $f^* = f(x, \omega^*)$ satisfies:

$$\int \frac{\partial L}{\partial f} p(y | x) dy = 0$$

Summary (2)

Binary Cross Entropy

- Training with the binary cross entropy loss yields a classifier that approximates the class probability.
- This fact can be exploited in creative ways. Two examples were given.